

Esquemas Recursivos No Final y Múltiple

**Análisis y Diseño de Datos y Algoritmos
Estructuras de Datos y Algoritmos**

**ESCUELA TÉCNICA SUPERIOR DE
INGENIERÍA INFORMÁTICA
Departamento de Lenguajes y Sistemas Informáticos
Curso 2025-2026**

Elementos de los algoritmos recursivos

Esquema recursivo no final y algoritmos iterativos

Esquemas recursivos múltiples

Diseño de algoritmos recursivos

Recursividad no final

Elementos recursivos

Algoritmo recursivo final:

$$\text{mcd}(a, b) = \begin{cases} a, & b = 0 \\ \text{mcd}(b, a \% b), & a, b > 0 \end{cases}$$

Algoritmo recursivo simple no final:

$$n! = \begin{cases} 1, & n = 0 \\ n * (n - 1)!, & n > 0 \end{cases}$$

Algoritmo recursivo múltiple:

$$\text{fib}(n) = \begin{cases} n, & 0 \leq n \leq 1 \\ \text{fib}(n - 1) + \text{fib}(n - 2), & n > 1 \end{cases}$$

Recursividad no final

Esquema general:

```
R seqCollectRight(X x) {  
    B b = seqCollectRight(x, e0)  
    return r(b)  
}  
R seqCollectRight(X x, E e) {  
    B r = b0;  
    if(g(e)){  
        B r = seqCollectRight(x, nx(e));  
        if(!d(r)){  
            r = c(r,e);  
        }  
    }  
    return r;  
}
```

No hay algoritmo iterativo directamente equivalente, pero a veces se puede buscar uno.

Recursividad no final

Conceptos fundamentales: caso base, caso recursivo y tamaño de problema

Ejemplo: factorial

$$n! = \begin{cases} 1, & n = 0 \\ n * (n - 1)!, & n > 1 \end{cases}$$

```
Long f(Integer n) {  
    Long r;  
    if (n == 0) {  
        r = 1L;  
    } else {  
        r = n * f(n - 1);  
    }  
    return r;  
}
```

NO FINAL

$$f(n, a) = \begin{cases} a, & n = 0 \\ f(n - 1, n * a), & n > 1 \end{cases}$$

$$n! = f(n, 1)$$

```
Long f(Integer n) {  
    return f(n, 1);  
}  
Long f(Integer n, Long b) {  
    if (n > 0) {  
        b = f(n - 1, n * b);  
    }  
    return b;  
}
```

FINAL

Recursividad no final

Ejemplo: dada una lista, obtener su inversa

```
List<E> inverseRec(List<E> ls) {  
    return inverse(ls, 0);  
}  
List<E> inverseRec(List<E> ls, Integer i) {  
    List<E> r;  
    if (i < ls.size()) {  
        r = inverseRec(ls, i+1);  
        r.add(ls.get(i));  
    } else {  
        r = new ArrayList<E>();  
    }  
    return r;  
}
```

```
List<E> inverseWhile(List<E> ls) {  
    Integer i = 0;  
    List<E> r = new ArrayList<E>();  
    while (i < ls.size()) {  
        r.add(0, ls.get(i));  
        i++;  
    }  
    return r;  
}
```

Recursividad no final

Ejemplo: comprobar si una lista está ordenada

```
// Versión 1, no final. Importa el orden cuando se usa &&
Boolean isSortedRecNF(List<E> ls, Comparator<E> cmp) {
    return isSorted(ls, cmp, 0);
}
Boolean isSortedNF(List<E> ls, Comparator<E> cmp, Integer i) {
    Boolean r = true;
    Integer n = ls.size();
    if (i < n-1) {
        r = lesserEq(cmp, ls.get(i), ls.get(i+1))
            && isSorted(ls, cmp, i+1);
    }
    return r;
}
```

```
Boolean lesserEq(Comparator<E> cmp, E e1, E e2) {
    return cmp.compare(e1, e2) <= 0;
}
```


Recursividad no final

Ejemplo: comprobar si una lista está ordenada

```
// Versión 2, final. Se hace explícito el acumulador
Boolean isSortedRecF(List<E> ls, Comparator<E> cmp) {
    return isSorted(ls, cmp, 0, true);
}
Boolean isSortedF(List<E> ls, Comparator<E> cmp, Integer i, Boolean b) {
    Integer n = ls.size();
    if (i < n-1 && b) {
        b = lesserEq(cmp, ls.get(i), ls.get(i+1));
        b = isSorted(ls, cmp, i+1, b);
    }
    return b;
}
```

Recursividad no final

Ejemplo: comprobar si una lista está ordenada

```
// Versión funcional con el acumulador allMatch
Boolean isSortedFun(List<E> ls, Comparator<E> cmp) {
    return IntStream.range(0, ls.size()-1)
        .allMatch(i -> lesserEq(cmp, ls.get(i), ls.get(i+1)));
}
```

```
// Versión iterativa derivada de la versión final 2
Boolean isSortedWhile(List<E> ls, Comparator<E> cmp) {
    Integer i = 0;
    Boolean b = true;
    Integer n = ls.size();
    while (i < n-1 && b) {
        b = lesserEq(cmp, ls.get(i), ls.get(i+1));
        i++;
    }
    return b;
}
```

Ejemplo de transformaciones

Ejemplo: dada la siguiente definición recursiva (que toma como entrada 2 números enteros positivos y devuelve una cadena):

$$f(a, b) = \begin{cases} "(" + toString(a * b) + ")", & a < 5 \vee b < 5 \\ toString(a + b) + f(a/2, b - 2), & \text{en otro caso} \end{cases}$$

siendo + un operador que representa la concatenación de cadenas, y *toString(i)* un método que devuelve una cadena a partir de un entero, proporcionar una solución iterativa usando while, una recursiva no final, una recursiva final, y una en notación funcional.

Ejemplo de transformaciones

$$f(a, b) = \begin{cases} "(" + toString(a * b) + ")", & a < 5 \vee b < 5 \\ toString(a + b) + f(a/2, b - 2), & \text{en otro caso} \end{cases}$$

Solución recursiva no final (igual que la definición):

```
String fRecNF(Integer a, Integer b) {  
    String r = null;  
    if (a < 5 || b < 5) {  
        r = String.format("(%d)", a * b);  
    } else {  
        r = String.format("%d", a + b) + solicitudRecursivaNoFinal(a/2, b-2);  
    }  
    return r;  
}
```

Ejemplo de transformaciones

$$f(a, b) = \begin{cases} "(" + \text{toString}(a * b) + ")", & a < 5 \vee b < 5 \\ \text{toString}(a + b) + f(a/2, b - 2), & \text{en otro caso} \end{cases}$$

Solución recursiva final:

```
String fRecF(Integer a, Integer b) {  
    return fRecF(a, b, "");  
}  
String fRecF(Integer a, Integer b, String ac) {  
    String r = null;  
    if (a < 5 || b < 5) {  
        r = ac + String.format("(%d)", a * b);  
    } else {  
        r = fRecF(a/2, b-2, ac + String.format("%d", a + b));  
    }  
    return r;  
}
```

Ejemplo de transformaciones

$$f(a, b) = \begin{cases} "(" + \text{toString}(a * b) + ")", & a < 5 \vee b < 5 \\ \text{toString}(a + b) + f(a/2, b - 2), & \text{en otro caso} \end{cases}$$

Solución iterativa:

```
String fWhile(Integer a, Integer b) {  
    String ac = "";  
    // En la versión Rec, la condición era de terminación  
    while(!(a < 5 || b < 5)) {  
        ac += String.format("%d", a + b);  
        a /= 2;  
        b -= 2;  
    }  
    return ac + String.format("(%d)", a * b);  
}
```

Ejemplo de transformaciones

$$f(a, b) = \begin{cases} "(" + toString(a * b) + ")", & a < 5 \vee b < 5 \\ toString(a + b) + f(a/2, b - 2), & \text{en otro caso} \end{cases}$$

Solución
funcional:

```
private static record Tupla (Integer a, Integer b, String ac) {  
    public static Tupla first(Integer a, Integer b) {  
        return new Tupla(a, b, "");  
    }  
    public Tupla next() {  
        Integer newA = a; Integer newB = b; String newAc = ac;  
        newAc += String.format("%d", newA + newB);  
        newA /= 2;  
        newB -= 2;  
        return new Tupla(newA, newB, newAc);  
    }  
    public Boolean hasNext() {  
        return !(a < 5 || b < 5);  
    }  
    public String result() {  
        return ac + String.format("(%d)", a * b);  
    }  
}  
  
public static String fFun(Integer a, Integer b) {  
    return Stream.iterate(Tupla.first(a, b), t -> t.next())  
        .dropWhile(t -> t.hasNext()).findFirst().get().result();  
}
```

Recursividad múltiple

Elementos recursivos

Algoritmo recursivo final:

$$\text{mcd}(a, b) = \begin{cases} a, & b = 0 \\ \text{mcd}(b, a \% b), & a, b > 0 \end{cases}$$

Algoritmo recursivo simple no final:

$$n! = \begin{cases} 1, & n = 0 \\ n * (n - 1)!, & n > 0 \end{cases}$$

Algoritmo recursivo múltiple:

$$\text{fib}(n) = \begin{cases} n, & 0 \leq n \leq 1 \\ \text{fib}(n - 1) + \text{fib}(n - 2), & n > 1 \end{cases}$$

Esquema recursivo multiple

Esquema general de un algoritmo recursivo múltiple:

```
R f(x) {  
    if (b(x)) {  
        s = sb(x);  
    } else {  
        y1 = sp1(x);  
        s1 = f(y1);  
        y2 = sp2(x);  
        s2 = f(y2);  
        ...  
        s = c(s1, s2, ..., x);  
    }  
    return s;  
}
```

$b(x) \rightarrow$ es caso base?

$sb(x) \rightarrow$ solución del caso base

$y1, y2, \dots \rightarrow$ subproblemas

$sp1(x), sp2(x), \dots \rightarrow$ cálculo de los subproblemas

$s1, s2, \dots \rightarrow$ soluciones de los subproblemas

$c(s1, s2, \dots, x) \rightarrow$ función de combinación (soluciones de los subproblemas + propiedades del problema)

Esquema con memoria

Recursividad con memoria:
para evitar cálculos cuando
hay muchos subproblemas
repetidos

Necesitamos un diccionario
(m) que almacene la solución
 s para cada problema x ya
resuelto

P es el tipo de los problemas
y S es el tipo de la solución

```
S fM(x) {  
    Map<P,S> m = ...;  
    return fM(x, m);  
}  
S fM(x, m) {  
    ...  
    if (x in m.keys()) {  
        s = m[x];  
    } else if (b(x)) {  
        s = sb(x);  
        m[x] = s;  
    } else {  
        y1 = sp1(x);  
        s1 = fM(y1, m);  
        y2 = sp2(x);  
        s2 = fM(y2, m);  
        ...  
        s = c(s1, s2, ... , x);  
        m[x] = s;  
    }  
    return s;  
}
```

Fibonacci sin memoria

```
BigInteger fibRec(Integer n) {  
    BigInteger r;  
    if (n == 0) {  
        r = BigInteger.ZERO;  
    } else if (n == 1) {  
        r = BigInteger.ONE;  
    } else {  
        r = fibRec(n-1).add(fibRec(n-2));  
    }  
    return r;  
}
```

Fibonacci con memoria

```
BigInteger fibRecM(Integer n) {  
    return fibRecM(n, new HashMap<>());  
}  
BigInteger fibRecM(Integer n, Map<Integer, BigInteger> m) {  
    BigInteger r;  
    if (m.containsKey(n)) {  
        r = m.get(n);  
    } else if (n == 0) {  
        r = BigInteger.ZERO;  
        m.put(n, r);  
    } else if (n == 1) {  
        r = BigInteger.ONE;  
        m.put(n, r);  
    } else {  
        r = fibRecM(n-1, m).add(fibRecM(n-2, m));  
        m.put(n, r);  
    }  
    return r;  
}
```

Transformación recursivo múltiple a iterativo

Trasformación recursivo múltiple a iterativo

$$fb(n) = \begin{cases} d_0, & n = 0 \\ d_1, & n = 1 \\ a * fb(n-1) + b * fb(n-2), & n > 1 \end{cases}$$

$e=(i, u, v) \rightarrow ei=(0,d1,d0)$

I: $(i,u,v)=(i,fb(i+1),fb(i))$

$(i+1,u',v')=(i+1,fb(i+2),f(i+1))$
 $\quad\quad\quad=(i+1,a*fb(i+1)+b*fb(i),f(i+1))$
 $\quad\quad\quad=(i+1,a*u+b*v, u')$

Fin: $i=n-1 \Rightarrow ef=(n-1,fb(n),f(n-1))$

Trasformación recursivo múltiple a iterativo

$e=(i,u,v) \rightarrow ei=(0,d1,d0)$

I: $(i,u,v)=(i,fb(i+1),fb(i))$

$$\begin{aligned}(i+1,u',v') &= (i+1,fb(i+2),f(i+1)) \\ &= (i+1,a*fb(i+1)+b*fb(i),f(i+1)) \\ &= (i+1,a*u+b*v, u')\end{aligned}$$

fin: $i=n-1 \Rightarrow ef=(n-1,fb(n),f(n-1))$

```
fblin(n,a,b,d1,d0) {  
    (i,u,v) = (0,d1,d0);  
    while(i < n) {  
        (i,u,v) = (i+1, u*a + v*b, u);  
    }  
    return v;  
}
```

```
fblin(n,a,b,d1,d0) {  
    i = 0;  
    u = d1;  
    v = d2;  
    while(i < n) {  
        i = i + 1;  
        u0 = u;  
        u = u * a + v * b;  
        v = u0;  
    }  
    return v;  
}
```


Transformación a iterativo

$$g(a, b) = \begin{cases} a + b^2, & a < 2 \wedge b < 2 \\ a^2 + b, & a < 2 \vee b < 2 \\ g\left(\frac{a}{2}, b - 1\right) + g\left(\frac{a}{3}, b - 2\right) + g\left(a - 2, \frac{b}{4}\right), & \text{en otro caso} \end{cases}$$

Siendo a y b enteros positivos



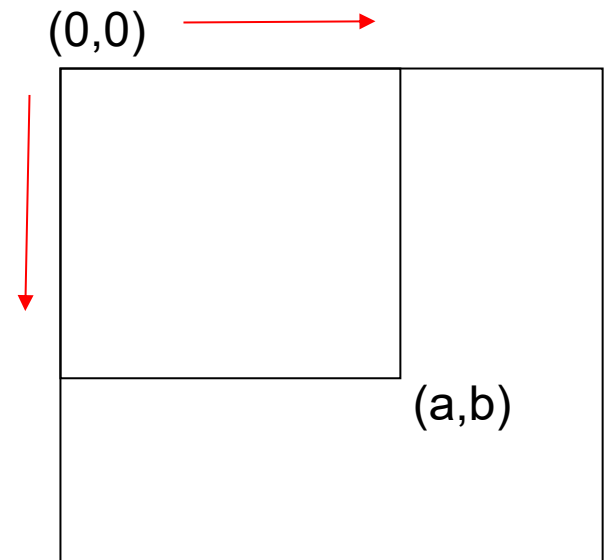
Estrategia bottom-up:

- Se empieza por los casos base
- Hay que calcular los subproblemas a partir de los casos base
- Los problemas a partir de los subproblemas

Transformación a iterativo

$$g(a,b) = \begin{cases} a + b^2, & a < 2 \wedge b < 2 \\ a^2 + b, & a < 2 \vee b < 2 \\ g\left(\frac{a}{2}, b-1\right) + g\left(\frac{a}{3}, b-2\right) + g\left(a-2, \frac{b}{4}\right), & \text{en otro caso} \end{cases}$$

```
Integer g(Integer a, Integer b) {
    Integer[][] m = {}
    Integer v = null;
    for (int i = 0; i <= a; i++) {
        for (int j = 0; j <= b; j++) {
            if (i < 2 && j < 2) {
                v = i + j * j;
            } else if (i < 2 || j < 2) {
                v = i * i + j;
            } else {
                v = m[i/2][j-1] +
                    m[i/3][j-2] +
                    m[i-2][j/4];
            }
            m[i][j] = v;
        }
    }
    return m[a][b];
}
```



Diseño de algoritmos

Diseño de alg. recursivos

Opción 1: Partir de una definición o propiedades recursivas

- definir tamaño del problema y casos base
- encontrar algoritmo

Opción 2: Generalizar el problema (añadir propiedades)

- definir tamaño y casos base
- reducir cada problema a subproblemas más pequeños
- definir la solución en base a la solución de los subproblemas

Ejemplo 1: Diseñar un algoritmo basado en estas propiedades

$$\begin{aligned}\binom{n}{0} &= \binom{n}{n} = 1, & n \geq 0 \\ \binom{n}{1} &= \binom{n}{n-1} = n, & n \geq 1 \\ \binom{n}{k} &= \binom{n}{n-k}, & n \geq k \\ \binom{n}{k} + \binom{n}{k+1} &= \binom{n+1}{k+1}, & n \geq k\end{aligned}$$

Diseño de alg. recursivos

Opción 1: Partir de una definición o propiedades recursivas

- definir tamaño del problema y casos base
- encontrar algoritmo

Opción 2: Generalizar el problema (añadir propiedades)

- definir tamaño y casos base
- reducir cada problema a subproblemas más pequeños
- definir la solución en base a la solución de los subproblemas

Ejemplo 2: Diseñar un algoritmo para encontrar el elemento más cercano a otro en una lista ordenada.

A partir de propiedades

$$\binom{n}{k} = \begin{cases} 1, & k, n - k = 0 \\ n, & k, n - k = 1 \\ \binom{n-1}{k-1} + \binom{n-1}{k}, & k > 1 \end{cases}$$

```
BigInteger binom(Long n, Long k) {  
    BigInteger r;  
    if (k == 0 || n-k == 0)  
        r = BigInteger.ONE;  
    else if (k == 1 || n-k == 1)  
        r = BigInteger.valueOf(n);  
    else  
        r = binom(n-1, k-1).add(binom(n-1, k));  
    return r;  
}
```

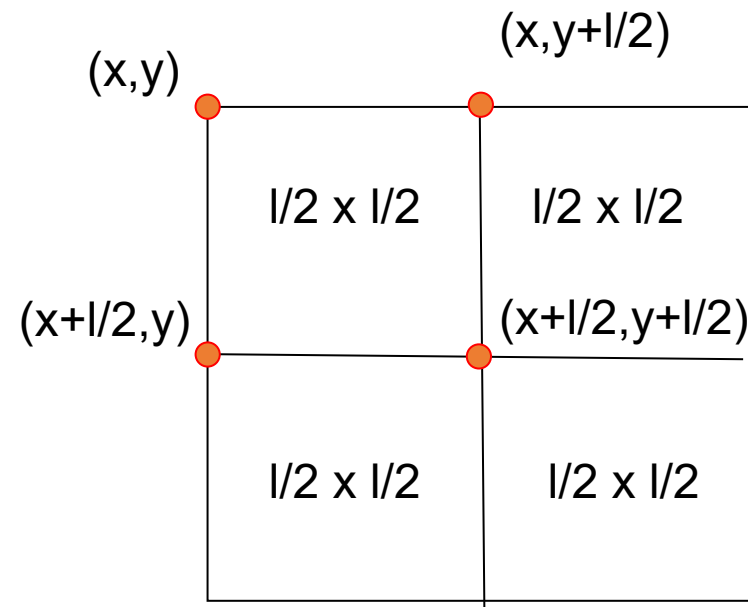
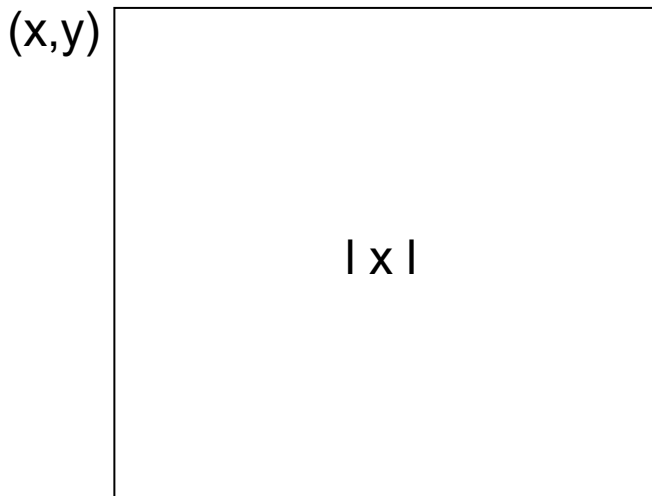
Recursividad múltiple y matrices

Recursividad múltiple y matrices

2. Diseñe un algoritmo que dada una matriz de $n \times n$ enteros (con $n=2^m$; siendo m un número entero mayor que 0), devuelva cierto si el valor en la casilla superior izquierda es menor al de la casilla inferior derecha, y cada una de las 4 submatrices cumplen también dicha propiedad, y así sucesivamente.

Generalización del problema (x,y,l) :

- x,y : Esquina superior izquierda de la submatriz
- l : Tamaño del lado de la submatriz



Recursividad múltiple y matrices

Matrix<E>

```
Boolean f(Integer x, Integer y, Integer l, Integer[][] m) {  
    Boolean r = null;  
    if (l == 2) {  
        r = m[x][y] < m[x+l][y+l];  
    } else {  
        r = m[x][y] < m[x+l][y+l] &&  
            f(x, y, l/2, m) &&  
            f(x+l/2, y, l/2, m) &&  
            f(x, y+l/2, l/2, m) &&  
            f(x+l/2, y+l/2, l/2, m);  
    }  
    return r;  
}
```

Esquemas Recursivos No Final y Múltiple

**Análisis y Diseño de Datos y Algoritmos
Estructuras de Datos y Algoritmos**

**ESCUELA TÉCNICA SUPERIOR DE
INGENIERÍA INFORMÁTICA
Departamento de Lenguajes y Sistemas Informáticos
Curso 2025-2026**